

The CodeInverter Suite: Structure- and Data-Aware Binary Decompilation with Efficient LLMs

Peipei Liu
Zhongguancun Laboratory
Beijing, China
liupp@zgclab.edu.cn

Jian Sun
Rongkang Sun
Zhongguancun Laboratory
Beijing, China
{[@zgclab.edu.cn](mailto:sunjian,shunrk2025)}

Li Chen*
Zhaoteng Yan*
Zhongguancun Laboratory
Beijing, China
{[@zgclab.edu.cn](mailto:chenli,yanzt)}

Xiaoling Zhang
Dawei Wang
Zhongguancun Laboratory
Beijing, China

Dapeng Sun
Peizheng Zhang
Zhongguancun Laboratory
Beijing, China

Dan Li
Tsinghua University
Beijing, China
tolidan@tsinghua.edu.cn

Abstract

Binary decompilation plays a vital role in various cybersecurity and software engineering tasks. Recently, end-to-end decompilation methods powered by large language models (LLMs) have attracted increasing attention for their ability to generate highly readable source code with minimal human intervention. However, existing LLM-based methods still struggle with reconstructing program structure and logic, achieving accurate data recovery, ensuring data security and privacy, and maintaining computational efficiency. To address these challenges, we propose the **CodeInverter Suite**, with three main pieces: (1) the CodeInverter Workflow (CIW) is a novel prompt engineering method that incorporates control flow graphs (CFG) and explicit data mappings to enhance structure reconstruction and data recovery during decompilation. (2) Building upon CIW, we construct the CodeInverter Dataset (CID), a large-scale domain-specific dataset containing 8.69 million samples enriched with CFGs and data mapping information. (3) We develop CodeInverter Models (CIMs), two lightweight LLMs with 1.3B and 6.7B parameters, enabling efficient inference in privacy-sensitive and resource-constrained environments. Extensive experiments on two benchmark datasets demonstrate that CIW significantly enhances the decompilation performance of various LLMs, with average improvements of 13.07% in re-executability and 23.94% in re-compilability. For our proposed decompilation model, CIM-6.7B achieves state-of-the-art performance in terms of re-executability and readability, outperforming existing LLMs—even with over 100× more parameters—by an average of 11.03% and 6.27%, respectively.

CCS Concepts

• **Software and its engineering** → **General programming languages**; • **Security and privacy** → **Software reverse engineering**.

*Corresponding authors: Li Chen and Zhaoteng Yan.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
<https://doi.org/10.1145/3832783.3837554>

Keywords

control-flow graph, data mapping, decompilation, large language models, software security

ACM Reference Format:

Peipei Liu, Jian Sun, Rongkang Sun, Li Chen, Zhaoteng Yan, Xiaoling Zhang, Dawei Wang, Dapeng Sun, Peizheng Zhang, and Dan Li. 2026. The CodeInverter Suite: Structure- and Data-Aware Binary Decompilation with Efficient LLMs. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3832783.3837554>

1 Introduction

Decompilation is a process that restores compiled binary code into functionally equivalent, human-readable high-level programming language code (i.e., **pscode**¹) [7, 12, 23]. It plays a critical role in numerous software security and digital forensics tasks [5, 9, 19, 30]. For example, when identifying software vulnerabilities or analyzing malware, security researchers often first convert binary executables into interpretable high-level representations due to the absence of source code.

Currently, end-to-end decompilation approaches with LLMs are gaining increasing attention for generating more readable and syntactically consistent pscode with reduced manual intervention and strong potential for rapid advancement. WaDec [29] first employs an LLM to decompile WebAssembly. LLM4Decompile [31] supports both direct binary decompilation and Ghidra [14] output refinement, achieving strong results in readability and executability. Nova [20] uses hierarchical attention and contrastive learning to enhance accuracy. FAE [11] recompiles decompiled code for in-context learning and better alignment with source code.

Despite impressive progress, there still remains practical challenges: (1) They model assembly instructions as plain text sequences, without explicitly capturing the inherent program structure. This makes it difficult to capture non-local topological constraints in program execution paths, particularly when handling complex control logic (e.g., nested conditional branches, indirect jumps, and loops), cross-block data propagation, and context-dependent relationships, leading to structural fragmentation and logical inconsistencies. (2)

¹In general, the recovered code is expressed in pseudo-C form. We use the term **pscode** as an abbreviation throughout the paper.

(a)

```

1 int resize_buffer(data_container *container,
2                  size_t new_size)
3 {
4     char *new_buffer = realloc(container->buffer,
5                                new_size);
6     if (new_buffer == NULL)
7         return -1;
8     container->buffer = new_buffer;
9     /* Forgetting to update the buffer size
10      may trigger a vulnerability! */
11     /* container->buffer_size = new_size; */
12     return 0;
13 }
14
15 int append_data(data_container *container,
16               const char *data, size_t data_len)
17 {
18     if (data_len > container->buffer_size) {
19         printf("Error: data too long\n");
20         return -1;
21     }
22     /* Buffer overflow may happen here! */
23     strcpy(container->buffer, data, data_len);
24     return 0;
25 }

```

(b)

```

void suspicious_function(int param_1) {
1  if (param_1 == 0) goto LAB_00123456;
2  if (param_1 < 3) goto LAB_00123478;
3  if (param_1 == 3) goto LAB_00123490;
4
5  function_a();
6  goto LAB_00123500;
7
8  LAB_00123456:
9  function_b();
10 goto LAB_00123500;
11
12 LAB_00123478:
13 function_c();
14 goto LAB_00123500;
15
16 LAB_00123490:
17 function_d();
18
19 LAB_00123500:
20 cleanup_function();
21 return;
22 }

```

Figure 1: (a): The vulnerability reappears in an LLM’s decompilation, (b): Excessive use of GOTO in Ghidra’s decompilation

They overlook the data context suggested by memory addresses in assembly instruction sequences, hindering the accurate recovery of data objects and their usage relationships during decompilation. For example, in `mov eax, [rip+0x3fa29](rip=0x4729)`, the model may treat `[rip+0x3fa29]` merely as a token, ignoring the actual data information stored at the address `.bss:0x44152`.

Furthermore, some LLM-based decompilation works [18, 33] rely on general-purpose LLMs, including both open-source (e.g., LLaMA [32]) and commercial models (e.g., GPT[28]). However, achieving competitive decompilation performance with open-source LLMs requires prohibitively high computational resources and operational costs. For instance, performing inference with a 70B open-source LLM demands at least 4×48GB GPUs (with vLLM + FP16). On the other hand, utilizing third-party commercial models for decompiling binaries may pose significant data security and privacy risks, especially when the binaries contain proprietary algorithms, business-critical logic, or user data. For example, the LLM-generated code contained an identical vulnerability after we used the LLM for CVE analysis (as shown in Fig.1(a)).

To address the above issues, we propose the **CodeInverter Suite**, consisting of three pieces: the CodeInverter Workflow (CIW), the CodeInverter Dataset (CID), and the CodeInverter Models (CIMs).

The **CIW** is a novel prompt engineering method with two core ideas according to decompilation principles and properties: (1) CIW integrates the semantic modeling strengths of LLMs with the topological insights of control flow graphs (CFGs) for end-to-end decompilation. By transforming CFGs into sequence-friendly inputs, CIW explicitly incorporates control-flow information into LLM prompts, enabling accurate reconstruction of program structure and control logic. (2) To improve data object recovery, we enhance assembly semantics with an explicit data mapping table extracted from binary data sections such as `.rodata`, `.data`, `.bss`, and `call` stack layouts. The data mapping reflects relationships between raw memory addresses and their corresponding data, and this mechanism makes implicit data dependencies visible to the model and strengthens data object recovery. Guided by the CIW, we construct the **CID**, a large-scale dataset enriched with CFG and data mapping tables annotations, specifically tailored for decompilation. CID provides fundamentally different learning signals from previous work that better reflect deeper semantic modeling in decompilation, shifting the objective from surface-level instruction translation to semantics-aware decompilation. Trained on the CID, we achieve two decompilation LLMs with 1.3B and 6.7B parameters (**CIM-1.3B/6.7B**) to

accommodate privacy-sensitive and resource-constrained local deployment. Compared to prior work, our compact models not only offer several distinct advantages such as deployability, lower computational resource requirements, and complete security control but also significantly improve performance. Experimental results show our models achieve superior performance across major decompilation benchmarks (re-executability, and readability).

In this paper, our contributions can be summarized as follows:

- We reveal that control-flow structure loss and data-context absence are key factors limiting LLM-based decompilation, and propose CIW to systematically address both issues. To the best of our knowledge, CIW is the first framework to incorporate CFG and data mapping into LLMs, enabling effective knowledge alignment for the decompilation task.
- We construct the CID dataset, which contains 8.69 million function-level assembly–source code pairs for decompilation, each accompanied by semantic labels aligned with control-flow graphs and data-mapping tables for memory addresses. CID provides richer supervision signals that enable the model to better learn inductive biases consistent with decompilation principles.
- We have trained two lightweight decompilation LLMs CIM-1.3B and CIM-6.7B. Both models can be deployed locally with modest computational resources, ensuring strong data security and privacy.
- We demonstrate the effectiveness of the CIW and the superiority of the CIMs, and open-source the CIW design process, CID and CIMs to facilitate community research and applications at HuggingFace: <https://huggingface.co/CodeInverter>.

2 Background & Motivation

2.1 Control Flow Graph

CFG is a widely adopted intermediate representation in binary program analysis. It decomposes programs into basic blocks and explicitly models transfer relationships between them (e.g., conditional jumps, loops, and function calls), thereby characterizing all potential execution paths in a program. Compared to linear representations, CFGs provide more accurate topological structures and control dependency information from a static perspective, enabling models to better understand control constraints and dependency relationships between different code blocks.

CFGs have been widely explored in binary analysis. These works support tasks such as binary-code matching [35], similarity analysis [34], traditional decompiler pipelines [6], type inference [36], and procedure name prediction [7, 13] by modeling control flow, guiding data-flow analysis, and preserving execution context. However, to the best of our knowledge, few studies have explored using CFGs to assist LLMs in the decompilation process.

2.2 Decompilation

To achieve efficient and accurate decompilation, both the research community and industry have proposed a variety of approaches, which can be broadly categorized into following four categories: Traditional decompilers, such as Hex-Rays [15] and Ghidra [14], primarily rely on heuristic rules and pattern-matching strategies. Machine/Deep learning-enhanced methods[6, 7, 21, 22, 25, 36] integrate learning-based architectures to automatically learn mappings from low-level binaries to high-level source code. Hybrid methods

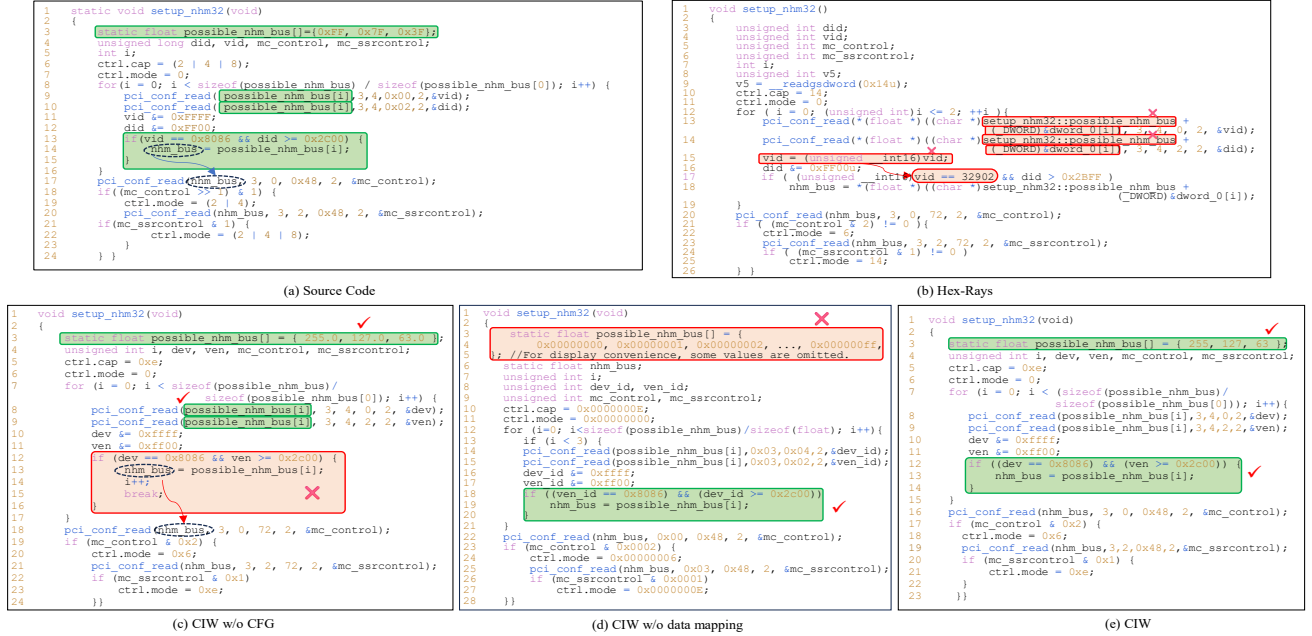


Figure 2: Motivating example. Presented are the (a) source code of the case sample alongside the decompilations of (b) Hex-Rays, (c) CIW without CFG on LLM, (d) CIW without data mapping on LLM and (e) CIW on LLM.

that combine LLM with traditional decompilers[18, 29, 33] first use traditional decompilers to generate pcode, and then use LLMs to improve and refine the results. End-to-end decompilation with LLMs[11, 20, 29, 31] directly convert assembly code² to pcode through prompt engineering or fine-tuning.

Although effective, these methods still have several limitations: **(I)** Traditional decompilers rely on static program analyses, heuristic rules, and pattern-matching strategies for high-level structure recovery, type recovery, and pcode generation. Developing and maintaining these components across architectures, compilers, and programming languages requires substantial engineering effort, limiting scalability. Moreover, the generated pcode often deviates from standard semantic abstractions and coding conventions (e.g., excessive use of GOTO in Fig.1(b)), reducing readability and increasing analysts’ cognitive burden. **(II)** Seq2Seq-based neural machine translation methods improve readability and extensibility, but limited model capacity often leads to unclear structural boundaries and inadequate modeling of target-language syntax. **(III)** Hybrid methods, while improving readability, remain fundamentally constrained by the intermediate representations produced by traditional decompilers. Consequently, they often inherit structural inaccuracies and semantic errors from the first stage, which the subsequent LLM cannot fully rectify, leading to semantically incorrect or incomplete code.

These limitations motivate the exploration of more flexible and scalable paradigms. In particular, end-to-end decompilation with LLMs has emerged as a promising direction, due to its ability to generate syntactically consistent and more readable pcode while offering efficient extensibility and maintainability.

In this work, we use IDA as the disassembly tool to facilitate the construction of CFGs and data mapping tables³, and subsequently perform end-to-end decompilation with LLMs.

2.3 Motivations

Given the proven utility of structural features derived from CFGs in binary analysis, and the increasing adoption of LLMs in decompilation tasks, we aim to explore the integration of CFGs with LLMs to improve the intelligence and effectiveness of the decompilation process. Beyond control flow, we further observe that explicitly mapping memory addresses to their corresponding data contexts can significantly enhance the recovery of data objects, especially in complex binary programs. Such mappings help reveal the relationships between variables, their types, and usage patterns—information that is often obscured or lost during compilation.

To clearly illustrate the motivation behind our work, we present a concrete example that shows the benefits of incorporating both control-flow structures and data-related contextual information into an LLM (DeepSeek-V3).

The source code of the motivating function `setup_nhm32` is in Fig.2(a). This function’s behavior relies on two critical elements: (1) the initialization of the variable `possible_nhm_bus` (line 3), which is propagated to its caller functions, and (2) the `if` branch at line 13, where computations influence subsequent control flow. When compiled with `-O0` for the x86-32 architecture and decompiled using the Hex-Rays[15] (Fig. 2(b)), we observe compound failures: the variable `possible_nhm_bus` remains uninitialized (lines 13-14), and `vid` is incorrectly assigned (line 15). These issues jointly corrupt the subsequent branch execution, illustrating the interdependence

²In binary research works, researchers typically use disassembly tools to first convert binary code into assembly code, which then serves as the foundation for downstream task-specific analyses.

³Although IDA is used in our implementation, the process is tool-agnostic and other disassembly tools can also be used to construct CFGs and data mapping tables.

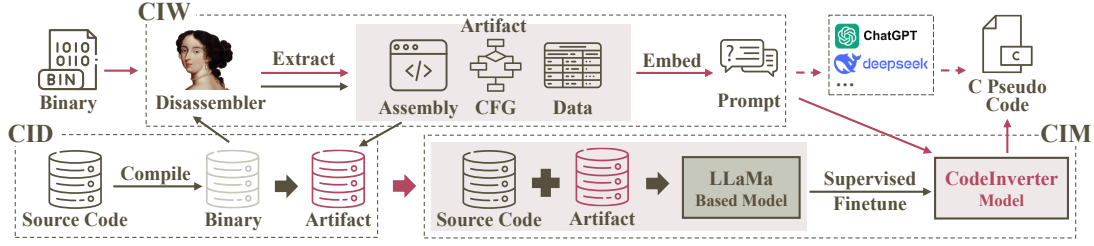


Figure 3: The decompilation workflow with LLMs in this paper (top: inference, bottom: training).

between value recovery and control-flow reconstruction—errors in one domain can propagate and compromise the other.

These limitations motivate the design of CIW. The data-mapping-only variant accurately recovers constants (Fig. 2(c)) but produces incoherent control structures (e.g., lines 14–15). In contrast, the CFG-only version (Fig. 2(d)) preserves the logical control flow but severely misrecovers values—for instance, expanding the variable `possible_nhm_bus` to 256 data entries, whereas the source defines only 3. These errors cascade: incorrect values distort the behavior of caller functions, and flawed control flow disrupts value propagation paths. Only the full CIW (Fig. 2(e)), which integrates both components, achieves functionally correct decompilation. While minor representational differences remain, the synergy of CFG-guided structural accuracy and precise value recovery resolves the interdependence problem observed in flat text prompt. This demonstrates that reliable decompilation requires the joint validation of macro-level control structures and micro-level data values—a capability uniquely enabled by our work.

3 Design of CodeInverter Suite

3.1 The Design of CIW

Unlike the input formulation adopted by LLM4Decompile[31]—where raw assembly instructions are concatenated with line-feed characters into a flat string [ASM code] (e.g., `mov eax, cs:cfg\nmov cs:aasworld, eax\nret`) and the LLM is prompted with a generic template “# This is the assembly code: [ASM code] # What is the source code?”—we carefully craft control-flow graph and data mapping and inject both information into the LLM prompt input. This approach effectively activates the LLM’s specific domain knowledge and guides it toward generating more reasonable and accurate decompilation results.

3.1.1 CFG-augmented Code Structure Reconstruction. In decompilation tasks, achieving code structure and control logic consistent with the original source code requires LLMs to effectively analyze and leverage features including branch logic, loop boundaries, and functional hierarchies in assembly instructions. However, raw assembly instructions are presented as linear sequences and lack explicit representations of these features, making it difficult for LLMs to accurately recover the source code. Considering that the CFG can explicitly model a program’s execution logic and provide LLMs with structured topological cues, we propose to integrate CFGs with LLMs to facilitate the code structure reconstruction.

CFG Extraction. Based on the assembly code obtained from the input binary, we extract the CFG for target functions using the

IDA [16], with assistance from objdump [4] to improve function boundary identification.

The process begins with function identification: We first use IDA to determine function boundaries. When symbol tables are available, we identify function entry points by matching function symbols with their corresponding addresses in the binary. In cases where IDA fails to identify a target function, we use the starting address derived from objdump to specify the entry point for subsequent control flow analysis.

After that, basic block segmentation is performed within each function recursively, where the instruction sequence is split at each control transfer instruction (e.g., jumps, calls) and their immediate targets. Then, edges between basic blocks are established through direct branch resolution for unconditional and conditional jumps, and through fall-through analysis for sequential execution paths.

CFG Representation. The extracted CFG is first represented as a dictionary incorporating well-labeled assembly instruction blocks (as shown in Figure 4), and then the dictionary is serialized into a JSON string to meet the input requirements of LLMs.

To improve both machine understanding and processing efficiency, we use a semantically enriched symbolic labeling scheme designed by [16]. Each basic block is assigned a unique symbolic label, and the target transfer addresses in control flow instructions are replaced with the corresponding block labels. For instance, the instruction `jg short 0x473a` is rewritten as `jg short loc_473A`, where `loc_473A` is the label assigned to the target block. Similarly, function call operands are replaced with function identifiers instead of numeric addresses, such as `puts_bench` in `loc_473A` block.

Subsequently, we represent the CFG in a dictionary (“Control Flow Graph” in Figure 4) consisting of three key fields: “nodenum” (the total number of basic blocks), “nodes” (the instruction content of each block), and “edges” (a list of directed edges between blocks). Each edge is represented as a pair of source and target block labels—for example, [“start”, “loc_4729”] indicates a control transfer from the start block to the `loc_4729` block.

3.1.2 Data Mapping-augmented Data Object Recovery. During the program compilation, most data objects—including variables and constants—are stored in designated non-code memory regions, with the exception of a small subset of constants that are embedded directly as immediate values within the code region `.text`. These memory regions are organized as follows: the `.rodata` section stores immutable data known at compile time, including string literals, constant arrays, and other `const`-qualified variables; the `.data` section contains global and static variables that are explicitly initialized with non-zero values; the `.bss` section holds global and static variables that are uninitialized in the source code, which are

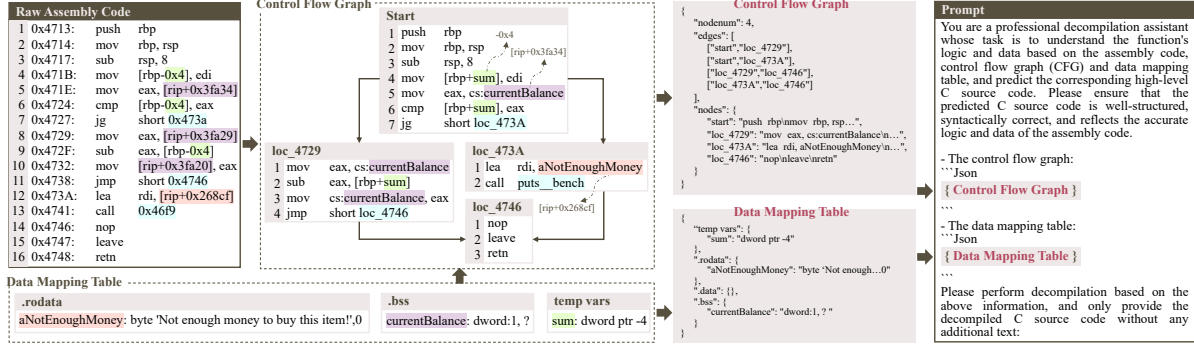


Figure 4: Prompt engineering details of our proposed CIW (the resulting prompt for all LLMs)

zero-initialized at runtime; the `call` stack holds temporary variables (e.g., local variables, function arguments) and return addresses, facilitating runtime context preservation and function invocation.

In binary decompilation, distinguishing between the `.data` and `.bss` sections aids in inferring variable types and recovering global/static data objects. The `.rodata` section contains both semantically rich data—such as string literals—and structural elements like jump tables, which are critical for reconstructing constant references and control-related data objects, respectively.

Therefore, leveraging these data sections can improve the completeness and accuracy of source-level data object recovery compared to prior `.text`-only methods. Although Ref Decompile [10] considers such sections, it relies on metadata extracted from the source code, making it impractical and unrealistic in real-world scenarios where the source code is unavailable. In contrast, we provide a more practical and principled solution by exploiting cues from the binary-level data sections.

Data Retrieval. We build an IDA plugin to retrieve the data associated with each target function. The process begins by identifying relevant data sections (e.g., `.rodata`, `.bss`) through parsing the section headers. These sections define the scope of static memory regions that may contain data used by the program.

After identifying these sections, we extract their full contents to form a pool of candidate data items for further analysis. We then perform cross-reference analysis to determine which data items are validly referenced in the assembly instructions. Data items that are explicitly referenced within the address range of the target function are retained as directly associated. For data items without explicit references, we additionally include those that are in close proximity to already retained items, under the assumption that such proximity may indicate memory adjacency (e.g., array elements). This heuristic helps reduce the risk of omitting relevant data that may be implicitly referenced.

Additionally, we identify temporary stack variables by analyzing stack register addressing and offset values, as they are crucial for recovering the types and semantics of underlying data objects.

Data Mapping Table. With the retrieved data, we subsequently replace the raw address identifiers with semantical symbol information available in the binary following [16], such as `[rip+0x3fa34]` (`rip=0x471E`) $\rightarrow$ `.bss:0x44152  $\rightarrow$  currentBalance` and `-0x4  $\rightarrow$  sum` (Figure 4). As a result, we obtain a data mapping table corresponds to the target function, assigning meaningful data items to the memory addresses in the assembly instructions.

As shown in Figure 4, each data item includes its size and corresponding value. Specifically, data sizes are denoted using standard types: `byte`, `word`, `dword` (double word), and `qword` (quad word); values are primarily represented in hexadecimal format, except when strings are detected—such as “*Not enough money to buy this item!*”. Strings are identified by scanning for sequences of printable ASCII or Unicode characters terminated by null bytes. Based on contextual and section attributes, appropriate string types (e.g., C-style or Pascal-style) are then assigned. Uninitialized items in the `.bss` section are denoted with a “?”. To concisely represent consecutive uninitialized items of the same size in the `.bss` section—e.g., array-like structures—a count is appended to the data size. For example, “`dword: 1, ?`” indicates a single uninitialized `dword`-sized value. For stack variables, only the offset and data size are recorded, such as `temp vars` in Figure 4.

Similar to the representation of the CFG, the data mapping table is structured as a dictionary and then serialized into a JSON string, allowing LLMs to effectively interpret and utilize the information.

3.1.3 Decompilation-Oriented Prompt Design. Through continuous testing and analysis, we have developed CIW, a highly structured prompt paradigm specifically designed for decompilation. By establishing the precise role of a “binary-aware decompilation expert”, it deeply integrates structural constraints from control flow graphs and semantic information from data mapping, effectively triggering the LLM’s capacity for deep reasoning and logical deduction. The design and test process of CIW is presented in the HuggingFace, and its content detail is shown in the Fig. 4.

CIW not only enhances the semantic alignment capability of the LLM, leading to more functionally accurate and semantically consistent outputs, but also repositions LLM-based decompilation work from “prompt engineering” to “structural prior modeling”. This establishes CIW as a bridge connecting traditional program analysis with LLMs.

3.2 CID Dataset Construction

We construct the CID dataset based on the ExeBench training dataset[1], a function-level source code dataset that pairs real-world C functions from GitHub with input-output (IO) examples to enable executable supervision. Starting with 1.2 million C functions from ExeBench training dataset, we generate our dataset with 32-bit and 64-bit through following steps (as illustrated in the bottom left of Figure 3), where different bits are for model generalization:

Step1: Resolve Data Type Conflicts. To generate 32-bit executables of C functions, we address data type conflicts between 32-bit standard libraries and Exebench-generated .c files. For instance, we replace the 64-bit-specific definition “typedef unsigned long size_t;” in Exebench with “typedef unsigned int size_t;” to comply with the 32-bit program standards.

Step2: Compile. The .c files containing the source code of the target functions are compiled using GCC into executables with four different optimization levels (O0, O1, O2, and O3). After compilation, the executables are stripped to remove debugging information for further processing.

Step3: Generate Disassembly Artifact. We disassemble the binary executables using IDA, and extract the corresponding CFGs and data mapping tables based on the processing techniques of CIW (i.e., Section 3.1.1 and Section 3.1.2).

Step4: Data Representation. Following the CIW paradigm, we integrate CFG and data mapping table to construct each training sample.

3.3 CIM Training

We adopt a sequence-to-sequence framework to train our decompilation LLM, which maps assembly instruction sequences to corresponding source code sequences. We initialize the model with checkpoints from LLM4Decompile [31], which is based on the LLaMA architecture [32]. The training objective follows the standard approach used in neural machine translation, which is well-suited for decompilation tasks due to their sequential generation nature: given an input sequence, the model learns to predict the corresponding output sequence token by token.

Output Prediction Probability. Given a CIW input sequence $\mathbf{x}_{in} = [x_1, \dots, x_m]$ and a target sequence of source code $\mathbf{x}_{out} = [x_{m+1}, \dots, x_n]$, the model predicts the probability of i -th token autoregressively using a softmax distribution over the decoder’s output logits:

$$P(x_i | x_{1:i-1}; \theta) = \text{softmax}(\mathbf{W}_o \mathbf{h}_i^{\text{dec}})_{x_i}, \quad (1)$$

where $\mathbf{h}_i^{\text{dec}}$ is the decoder’s hidden state at position i , $\mathbf{W}_o$ is the output projection matrix, and θ denotes all trainable parameters. During training, we apply teacher forcing by conditioning the prediction of x_i on the ground-truth prefix $x_{1:i-1}$.

Loss Function. The model is trained by minimizing the cross-entropy loss between the predicted token distributions and the ground-truth source code tokens. For a sequence of total length n , the loss is defined as:

$$\mathcal{L}(\theta) = - \sum_{i=m+1}^n \log P(x_i | x_{1:i-1}; \theta), \quad (2)$$

where the summation starts at $i = m + 1$ to ensure that only the target source code tokens contribute to the loss. This is equivalent to maximizing the conditional likelihood of the target sequence $\mathbf{x}_{out}$ given the input sequence $\mathbf{x}_{in}$ under the model parameters θ .

4 Experiments

We comprehensively evaluate our work against popular decompilers and SOTA decompilation LLMs across three metrics on two test dataset, and further dissect the contribution of each module. The experiments are conducted to address following research questions:

- RQ1: How effective is our proposed CIW approach?
- RQ2: How does CIM compare to existing LLMs and traditional decompilers in terms of effectiveness?
- RQ3: How effective are the individual components of CIW?
- RQ4: Is data-flow information necessary for achieving accurate decompilation?
- RQ5: What are the computational costs of using LLMs for binary decompilation?

4.1 Experimental Setups

4.1.1 Evaluation Metrics. Following prior work [2, 20, 31], we adopt following evaluation metrics:

Re-compilation Rate: This metric evaluates whether the decompiled code generated by our model can be successfully recompiled into an executable binary without errors. A high Recompilation Rate indicates that the decompiled code is syntactically correct and adheres to the constraints of the target programming language.

Re-executability Rate: This metric evaluates the functional correctness of the decompiled code. It measures whether the recompiled binary can execute successfully and produce outputs that accurately match the expected results. A high Re-executability indicates that the decompiled code is semantically and functionally correct, preserving the original program’s logical behavior.

The **Edit Similarity** is a key metric for evaluating the **readability** of decompiled code, and it quantifies the semantic similarity between the decompiled code and original code. Following [2, 17, 21], we next define normalized edit similarity ($ES(A, B)$), where higher scores indicate better readability of the decompiled code. Specifically, it is calculated as follows:

$$ES(A, B) = 1 - \frac{ED(A, B)}{L_B} \quad (3)$$

where A represents the prediction sequence, B represents the true sequence, L represents the length of the sequence, and $ED(A, B)$ represents the editing distance between A and B . Editing distance is also called Levenshtein distance, which means the minimum number of operations required to convert a sequence A to B . It is usually computed by dynamic programming, and its state transition equation is as follows:

$$ED(A_i, B_j) = \begin{cases} \max(L_A, L_B) & \min(L_A, L_B) = 0 \\ \min \begin{cases} ED(A_{i-1}, B_j) + 1 \\ ED(A_i, B_{j-1}) + 1 \\ ED(A_{i-1}, B_{j-1}) + 1 \end{cases} & \text{otherwise} \end{cases} \quad (4)$$

4.1.2 Baselines. To demonstrate the effectiveness of **CIW** and the superiority of **CIM**, we conduct vertical and horizontal evaluations. The vertical evaluation is designed to highlight the importance of **CIW** in the LLM-driven decompilation, while the horizontal evaluation compares the performance of our LLMs with baselines.

Based on this setting, we select general-purpose LLMs with strong generalization abilities (e.g., GPT-4o [27], DeepSeek-V3 [8], and Qwen-plus [3]), along with decompilation-specific LLMs (e.g., LLM4Decompile [31] and FAE [11]) as baselines. In addition, we select two traditional decompilers Hex-Rays [15] and Ghidra [14] as baselines. More details of the LLMs are in Supplementary Content A.1.2.

Table 1: Decompile results on HumanEval with only assembly instruction (without CIW). (%)

Metric	Model	HumanEval 32-bit					HumanEval 64-bit				
		O0	O1	O2	O3	AVG	O0	O1	O2	O3	AVG
Re-com	GPT-4o	85.98	82.32	83.54	78.05	82.47	89.02	77.44	85.98	79.27	82.93
	Deepseek-V3	94.51	87.80	85.98	89.63	89.48	95.73	88.41	89.02	84.15	89.33
	Qwen-plus	25.61	37.80	41.46	46.34	37.80	73.17	58.54	61.69	62.20	63.90
Re-exe	GPT-4o	22.56	11.59	13.41	9.51	14.27	33.54	10.98	14.02	9.76	17.08
	Deepseek-V3	57.93	29.27	33.54	35.37	39.03	66.46	36.59	37.80	37.80	44.66
	Qwen-plus	0.00	3.05	4.27	4.88	3.05	19.51	7.93	5.49	7.93	10.22
ES	GPT-4o	34.92	31.54	31.37	29.88	31.93	39.02	30.02	31.88	29.69	32.65
	Deepseek-V3	42.65	34.16	34.29	33.17	36.07	44.58	34.14	35.75	33.84	37.08
	Qwen-plus	10.64	15.78	16.42	15.71	14.64	27.44	22.06	21.57	22.29	23.34

Table 2: Comparisons between our CIMs with the baselines on HumanEval. *: Our reproduction using LLM4Decompile’s original settings. †: Results are from the original paper. (%)

Metric	Model	HumanEval 32-bit					HumanEval 64-bit				
		O0	O1	O2	O3	AVG	O0	O1	O2	O3	AVG
Re-com	GPT-4o +CIW	95.73	91.46	92.07	93.90	93.29	97.56	91.46	94.51	84.76	92.07
	Deepseek-V3 +CIW	96.34	92.07	89.63	90.85	92.22	97.56	87.80	85.37	78.66	87.35
	Qwen-plus +CIW	90.24	86.59	85.98	91.46	88.57	89.63	87.20	87.20	70.12	83.54
	Ghidra	21.95	30.49	25.61	28.66	26.68	21.95	19.51	15.85	15.24	18.14
	Hex-rays	29.88	35.37	35.37	34.15	33.69	26.22	21.34	26.83	25.61	25.00
	LLM4Decompile 1.3B*	9.15	25.00	17.68	18.90	17.68	56.10	58.54	54.27	56.10	56.25
	LLM4Decompile 6.7B*	7.32	34.15	35.98	35.98	28.35	71.95	80.49	75.00	75.00	75.61
	FAE†	-	-	-	-	-	92.07	93.29	92.07	93.90	92.84
	CIM-1.3B +CIW	85.98	87.20	90.24	89.36	88.26	90.85	87.80	87.80	86.59	88.26
Re-exe	CIM-6.7B +CIW	89.02	90.85	90.24	92.07	90.55	93.29	91.46	93.29	92.07	92.53
	GPT-4o +CIW	32.32	28.05	24.39	24.39	27.29	45.12	29.27	23.17	19.51	29.27
	Deepseek-V3 +CIW	54.27	40.24	34.76	34.15	40.86	71.34	45.73	45.73	42.07	51.22
	Qwen-plus +CIW	22.56	17.07	15.85	17.68	18.29	26.83	13.41	12.20	12.80	16.31
	Ghidra	7.32	12.20	11.59	12.20	10.83	19.51	14.63	12.20	11.59	14.48
	Hex-rays	19.51	27.44	27.44	26.83	25.31	23.17	17.07	21.95	20.73	20.73
	LLM4Decompile 1.3B*	0.00	1.22	0.61	0.61	0.61	25.61	10.37	7.32	9.76	13.26
	LLM4Decompile 6.7B*	0.61	1.22	0.61	0.61	0.76	39.02	26.22	30.49	28.05	30.94
	FAE†	-	-	-	-	-	71.95	53.66	48.78	45.73	55.03
ES	CIM-1.3B +CIW	65.24	34.15	35.37	31.71	41.62	71.34	39.63	42.07	40.24	48.32
	CIM-6.7B +CIW	75.61	53.05	53.05	50.00	57.93	80.49	57.93	56.71	53.05	62.05
	GPT-4o +CIW	40.81	35.47	36.28	35.41	36.99	44.48	36.45	36.86	34.11	37.98
	Deepseek-V3 +CIW	45.47	36.15	36.98	35.33	38.48	51.36	36.33	37.61	33.61	39.73
	Qwen-plus +CIW	39.71	34.80	35.76	34.37	36.16	43.59	35.75	35.14	31.63	36.53
	Ghidra	28.51	26.76	26.85	26.44	27.14	28.10	26.58	26.88	24.14	26.43
	Hex-rays	31.23	30.14	29.21	28.68	29.82	30.74	29.23	28.31	25.26	28.39
	LLM4Decompile 1.3B*	14.05	13.42	11.23	11.08	12.45	30.14	19.89	18.81	20.26	22.27
	LLM4Decompile 6.7B*	29.03	20.52	21.80	21.15	23.12	41.42	32.07	32.03	31.68	34.30
	CIM-1.3B +CIW	47.45	35.88	37.03	36.33	39.17	47.56	35.67	37.64	37.31	39.55
	CIM-6.7B +CIW	51.06	40.16	39.96	40.48	42.92	49.16	40.25	39.54	39.02	41.99

4.1.3 Evaluation Datasets. Following prior works [11, 20, 31], we use HumanEval [31] and ExeBench test set [1] as evaluation datasets. HumanEval consists of 164 real-world C functions and ExeBench test set contains 5,000 real-world C functions. Furthermore, we extend both test datasets to support 32-bit compilation for generalization testing.

Since HumanEval and ExeBench are publicly available benchmarks, we consider potential evaluation bias caused by benchmark exposure. We distinguish two possible sources of such bias.

(1) CIM may suffer from training-test contamination if the CID contains same functions from the evaluation datasets. To mitigate this risk, we perform multi-level source code decontamination between CID and the evaluation sets. First, we remove exact duplicate functions by computing hashes over normalized source code. Then, we perform AST-based deduplication. Specifically, we parse each source function into an AST and compute a hash value based on its normalized tree structure, where superficial textual differences

such as formatting and comments are removed. Training samples whose AST hashes match those of test samples are excluded.

(2) For general-purpose baseline LLMs, such as GPT-4o, DeepSeek-V3, and Qwen-plus, we cannot fully audit whether their pretraining corpora contain benchmarks. This uncertainty may affect absolute comparisons among independently pretrained LLMs. However, the comparison between CIM and LLM4Decompile provides a more controlled setting for analyzing potential benchmark exposure. CIM is initialized from the LLM4Decompile and is further trained on the decontaminated CID. Therefore, potential benchmark exposure inherited from the same base LLM is controlled in this comparison. After removing overlaps between CID and the evaluation benchmarks, the additional performance gain of CIM over LLM4Decompile can be attributed to the structure- and data-aware training signals introduced by CID and CIW, rather than direct benchmark memorization during fine-tuning.

Table 3: Decompile results on ExeBench with only assembly instruction (without CIW). (%)

Metric	Model	ExeBench-32-bit					ExeBench-64-bit				
		O0	O1	O2	O3	AVG	O0	O1	O2	O3	AVG
Re-com	GPT-4o	55.85	51.58	50.82	49.86	52.03	68.89	50.63	49.21	49.72	54.61
	Deepseek-V3	71.04	61.56	60.34	60.02	63.24	79.00	53.76	49.21	48.00	57.49
	Qwen-plus	43.43	31.85	31.55	32.86	34.92	57.85	42.11	44.59	39.82	46.09
Re-exe	GPT-4o	12.47	10.30	8.21	8.16	9.79	24.54	10.60	7.89	7.95	12.75
	Deepseek-V3	32.76	20.68	17.97	17.30	22.18	46.75	17.40	12.83	13.63	22.65
	Qwen-plus	7.87	1.63	1.41	0.79	2.93	17.69	2.19	4.05	3.17	6.78
ES	GPT-4o	36.59	30.59	29.58	29.70	31.62	41.88	30.90	29.62	29.10	32.88
	Deepseek-V3	45.22	35.14	33.21	32.84	36.60	50.30	35.00	32.77	32.25	37.58
	Qwen-plus	32.52	23.25	22.03	21.53	24.83	40.25	29.44	28.74	27.67	31.53

Table 4: Comparisons between our CIMs with the baselines on ExeBench. †: Results are from the original paper. (%)

Metric	Model	ExeBench 32-bit					ExeBench 64-bit				
		O0	O1	O2	O3	AVG	O0	O1	O2	O3	AVG
Re-com	GPT-4o +CIW	80.07	85.25	84.80	85.13	83.81	90.54	88.34	88.09	87.28	88.56
	Deepseek-V3 +CIW	84.30	81.30	80.15	81.41	81.79	93.32	89.68	89.98	88.68	90.42
	Qwen-plus +CIW	71.97	78.68	78.66	80.00	77.33	82.84	82.78	83.20	81.41	82.56
	Ghidra	68.49	62.48	56.71	56.80	61.12	68.48	69.80	67.10	65.86	67.81
	Hex-rays	65.10	57.32	48.57	44.79	53.94	56.96	57.88	51.71	49.96	54.13
	CIM-1.3B +CIW	84.14	73.55	74.72	73.28	76.42	88.33	70.76	70.72	69.99	74.95
	CIM-6.7B +CIW	86.87	73.77	73.01	73.52	76.79	89.49	70.57	70.20	68.14	74.60
Re-exe	GPT-4o +CIW	30.87	30.20	28.27	29.08	29.61	43.99	25.61	23.61	22.06	28.82
	Deepseek-V3 +CIW	43.83	36.55	34.26	34.51	37.29	59.16	36.48	32.89	30.86	39.85
	Qwen-plus +CIW	22.05	22.54	21.22	21.28	21.77	32.96	19.85	17.33	16.33	21.62
	Ghidra	61.48	57.42	44.91	44.45	52.06	63.79	66.31	57.43	58.53	61.51
	Hex-rays	57.76	50.63	39.04	37.72	46.29	54.67	52.99	43.05	37.34	47.01
	LLM4Decompile 1.3B†	-	-	-	-	-	17.86	13.62	13.20	13.28	14.49
	LLM4Decompile 6.7B†	-	-	-	-	-	22.89	16.60	16.18	16.25	17.98
	CIM-1.3B +CIW	56.09	35.33	33.94	32.49	39.46	65.20	36.32	33.34	32.55	41.85
	CIM-6.7B +CIW	64.74	42.86	40.60	40.66	47.21	72.13	40.42	36.57	35.45	46.14
ES	GPT-4o +CIW	52.02	40.53	38.68	38.44	42.42	46.53	41.24	39.85	40.16	41.95
	Deepseek-V3 +CIW	54.93	44.73	42.97	42.70	46.33	60.95	44.31	41.81	41.20	47.07
	Qwen-plus +CIW	43.62	40.39	39.15	39.06	40.56	50.98	39.87	38.59	37.70	41.79
	Ghidra	55.45	45.91	41.96	41.30	46.16	50.57	46.18	43.41	42.60	45.69
	Hex-rays	27.98	26.54	26.19	26.33	26.76	27.08	27.37	27.00	27.03	27.12
	CIM-1.3B +CIW	66.14	50.02	49.03	48.03	53.31	67.58	50.27	49.21	48.28	53.84
	CIM-6.7B +CIW	69.94	53.04	51.28	50.68	56.24	67.94	52.72	51.03	50.48	55.54

To further reduce the influence of benchmark-reuse bias, we additionally evaluate our CIM on an independent real-world application beyond existing benchmarks, as detailed in evaluation section 4.6.

4.1.4 Training and Inference Configuration. We train both 1.3B and 6.7B parameter LLM for one epoch (15 steps). The batch sizes are set to 16 and 12 for the 1.3B and 6.7B models, respectively. We employ the AdamW optimizer with an initial learning rate of $2e-5$, and set the maximum sequence length to 4096. Training is conducted on an NVIDIA GPU cluster with $8 \times H100-80GB$, and we utilize ColossalAI for efficient distributed training.

To enhance generation performance, we adopt a hierarchical training strategy, gradually introducing samples from simple to complex based on the number of CFG blocks. During inference and evaluation, we consistently apply greedy decoding. The decompilation process is accelerated using vLLM [24], which also ensures deterministic output.

4.2 Main Results

The main experimental results are presented in Table 1–2 for HumanEval, and Table 3–4 for ExeBench. Tables 1 and 3 report decompilation performance of general-purpose LLMs using our prompt

but only with raw assembly sequences. Tables 2 and 4 present the overall results, where general-purpose LLMs are enhanced with **CIW**, while decompilation-specific LLMs are either reproduced or directly cited from their original papers, depending on the availability of implementations. Results from traditional decompilers are also included for comparison. In the following, we will present some findings and analysis of the results.

4.2.1 The Effectiveness of CIW (RQ1). By comparing Table 1 with Table 2 and Table 3 with Table 4 through vertical evaluation, we observe that **CIW** significantly enhances decompilation performance on both HumanEval and ExeBench datasets. Specifically, on the HumanEval dataset, the three models achieve absolute improvements of 15.19% on Re-com, 9.16% on Re-exe, and 8.36% on ES; whereas on the ExeBench dataset, the corresponding improvements reach 32.67%, 16.98%, and 10.85%, respectively. Notably, meanwhile, Qwen-plus attains the largest improvement on Re-com (35.21%) and ES (17.36%) in HumanEval, while GPT-4o shows the largest improvement on Re-exe (12.61%); correspondingly, on the ExeBench dataset, Qwen-plus achieves the maximum enhancement on Re-com (39.44%) and ES (13.0%), and GPT-4o obtains the most substantial gain on Re-exe (17.95%).

Table 5: Ablation study for CFG. (%)

Metric	Model	ExeBench 64-bit					HumanEval 64-bit				
		O0	O1	O2	O3	AVG	O0	O1	O2	O3	AVG
Re-com	CIM-1.3B +CIW	87.34	65.02	62.70	62.35	69.35	91.77	88.54	89.17	87.90	89.35
	w/o cfg	87.02	63.14	61.44	61.18	68.19	90.51	90.45	90.45	89.17	90.15
	CIM-6.7B +CIW	87.23	62.75	59.77	61.81	67.89	93.04	92.99	94.27	92.99	93.32
Re-exe	w/o cfg	85.98	62.06	60.29	60.57	67.23	92.41	91.08	88.54	90.45	90.62
	CIM-1.3B +CIW	61.55	30.61	28.32	25.65	36.53	72.15	38.85	42.04	40.13	48.29
	w/o cfg	59.87	28.96	26.22	24.15	34.80	51.27	25.48	26.75	25.48	32.25
ES	CIM-6.7B +CIW	67.72	32.33	28.62	27.95	39.04	80.38	58.60	57.32	53.50	62.45
	w/o cfg	58.61	28.75	25.89	24.15	34.35	65.19	34.39	41.40	37.58	44.64
	CIM-1.3B +CIW	67.66	50.76	49.29	48.66	54.09	48.05	36.06	38.27	37.92	40.08
	w/o cfg	66.83	50.60	48.53	47.61	53.39	45.28	33.38	31.70	31.14	35.38
	CIM-6.7B +CIW	68.65	52.30	50.95	49.85	55.44	49.86	40.97	40.09	39.56	42.62
	w/o cfg	66.69	50.00	48.40	47.44	53.13	45.59	34.83	34.72	34.88	37.51

Table 6: Ablation study for data mapping table. (%)

Model	Metric	ExeBench 64-bit					HumanEval 64-bit				
		O0	O1	O2	O3	AVG	O0	O1	O2	O3	AVG
Re-com	CIM-1.3B +CIW	84.37	60.67	58.89	57.10	65.26	92.50	87.80	92.31	88.68	90.32
	w/o DM	80.18	60.39	58.33	58.45	64.34	95.00	90.24	92.31	90.57	92.03
	CIM-6.7B +CIW	86.86	62.64	60.56	59.25	67.33	87.50	92.68	94.87	94.34	92.35
Re-exe	w/o DM	84.03	56.74	59.44	61.39	65.40	95.00	90.24	84.62	84.91	88.69
	CIM-1.3B +CIW	55.83	23.31	14.72	13.40	26.82	67.50	31.71	38.46	33.96	42.91
	w/o DM	52.89	21.35	14.44	12.60	25.32	55.00	26.83	33.33	32.08	36.81
ES	CIM-6.7B +CIW	65.69	28.93	18.89	17.96	32.87	75.00	48.78	43.59	43.40	52.69
	w/o DM	60.93	22.47	17.50	17.43	29.58	55.00	41.46	41.59	41.51	44.89
	CIM-1.3B +CIW	68.43	47.89	39.43	38.28	48.51	42.94	33.10	36.59	36.32	37.24
	w/o DM	66.48	47.63	38.80	38.45	47.84	45.14	34.58	33.12	35.65	37.12
	CIM-6.7B +CIW	71.61	51.81	42.98	42.91	52.33	47.41	36.40	35.78	37.24	39.21
	w/o DM	70.38	48.24	44.18	42.28	51.27	48.16	37.76	35.02	37.51	39.61

Based on the observed performance improvements across different evaluation metrics demonstrated by various models on diverse datasets, we can confidently conclude that **CIW** plays a significantly positive role in decompilation. The consistent enhancement in performance metrics across multiple testing scenarios further demonstrates its robust generalization capability and strong adaptability to different code structures.

4.2.2 The Advancement of CIM Compared to other LLMs (RQ2). As shown in Tables 2 and 4, in the horizontal evaluation, our CIM-6.7B LLM achieves SOTA performance on Re-exe and the ES metric among LLM-driven decompilations, with average improvements of 11.03% and 6.27% (on two datasets), respectively, over DeepSeek-V3 671B—a model with 100× more parameters. This highlights our model’s strength in preserving semantic consistency and functional correctness. Although our Re-com score is slightly lower than that of some general-purpose LLMs, this is primarily because they possess stronger contextual modeling capabilities within more parameters. This allows them to generate syntactically complete code, even if it may not faithfully reflect the original program’s logic.

4.2.3 Comparing CIM with Traditional Decompilers (RQ2). From Table 2 and 4, we have the following insights: (1) On the Re-com metric, our CIMs significantly outperform traditional decompilers on both datasets. Specifically, the performance gap is most

substantial between CIM-6.7B (91.54%) and Ghidra (22.41%) on HumanEval, reaching 69.13%. In contrast, the gap is smallest between CIM-1.3B (75.69%) and Ghidra (64.47%) on ExeBench, at 11.22%. This indicates that LLMs surpass rule-based traditional decompilers in dimensions such as "semantic restoration" and "re-compilability", primarily due to their ability to effectively leverage internal knowledge to enhance the completeness of the generated code. (2) On the Re-exe metric, our CIMs and traditional decompilers shows a see-saw relationship. On the HumanEval dataset, even the sub-optimal CIM-1.3B (44.97%) outperforms the best-performing traditional decompiler, Hex-rays (23.02%). However, on the ExeBench dataset, the best-performing CIM-6.7B (46.68%) only matches the weakest-performing traditional decompiler, Hex-rays (46.65%), and is inferior to the best-performing traditional decompiler, Ghidra (56.79%). This suggests that while current LLMs offer greater robustness, a gap remains in core functionalities compared to developed decompilers. This is reasonable because traditional decompilers incorporate more targeted rules and domain knowledge, whereas CIM currently integrates only a few rules. Although the model itself may contain rich professional knowledge, more effective prompting methods are required to trigger and guide this knowledge. (3) On the ES metric, the CIMs perform excellently on both datasets, indicating a clear advantage of LLMs in the readability of decompiled output.

In summary, although LLM-driven decompilation approach still lags behind developed decompilers in certain aspects, our work demonstrates unique advantages across several key dimensions. We

Table 7: Decompilation results on HumanEval with CIW and DUG. (%)

Metric	Model	HumanEval 32-bit					HumanEval 64-bit				
		O0	O1	O2	O3	AVG	O0	O1	O2	O3	AVG
Re-com	GPT-4o +CIW +DUG	81.71	82.93	84.15	78.05	81.71	89.63	79.27	89.03	78.66	84.15
	Deepseek-V3 +CIW +DUG	92.65	92.67	92.08	93.89	92.82	92.07	89.63	92.07	92.08	91.46
	Qwen-plus +CIW +DUG	28.65	39.02	51.21	52.44	42.83	81.70	64.64	70.84	65.25	70.60
Re-exe	GPT-4o +CIW +DUG	18.29	7.32	10.97	7.07	10.91	25.61	10.98	14.63	6.10	14.34
	Deepseek-V3 +CIW +DUG	52.84	26.83	34.15	42.68	39.13	64.63	39.64	37.19	37.80	44.82
	Qwen-plus +CIW +DUG	0.00	6.71	10.37	4.88	5.34	24.39	13.42	7.93	4.88	12.66
ES	GPT-4o +CIW +DUG	34.43	30.84	31.31	30.41	31.75	37.83	31.27	31.08	29.72	32.48
	Deepseek-V3 +CIW +DUG	42.26	35.45	34.77	33.59	36.52	41.92	34.74	35.99	33.42	36.52
	Qwen-plus +CIW +DUG	10.46	15.27	17.54	15.55	14.71	29.03	22.33	21.77	22.04	23.79

hope that the LLM-driven decompilation paradigm will facilitate the integration of rules and knowledge from traditional decompilers with LLMs, thereby advancing the development of decompilation tools and making them more suitable for convenient deployment and application in complex scenarios.

4.2.4 The Impact of Compilation Optimization Levels. From the results across all four tables, we observe a clear trend across all models: decompilation performance generally degrades as the compiler optimization level increases (O0→O1→O2→O3). For example, on the Exebench 64-bit under the Re-exe metric, the average performance at O0 is 48.07%, O1 is 34.24%, O2 is 30.40%, and O3 is 29.18%. This decline is primarily due to the increasingly aggressive optimizations applied at higher levels (e.g., O2 and O3), such as expression rewriting, variable merging, dead code elimination, and control flow simplification. These transformations significantly alter the original function structure and reduce code readability, making it more challenging for models to accurately reconstruct the source code. Therefore, bridging the gap introduced by compiler optimizations and achieving more robust LLM-based decompilation remains an important direction for future work.

4.3 Ablation Study (RQ3)

To evaluate the contribution of individual components in our proposed **CIW**, we conduct an ablation study⁴ (also referred to as vertical experiments). Specifically, for a given LLM, we selectively remove either the CFG or the data mapping table (DM) from the input and observe the resulting impact on decompilation performance. The Tables 5 and 6 present the results⁵ of the ablation study on our CIMs. More results and analyses are shown in Supplementary Content A.2.

From the results, we have two key findings: (1) Overall, both CFG and DM improve performance in a complementary manner. Removing CFG causes average degradations of 10.07% on Re-exe, 3.21% on ES and 0.93% on Re-com; specifically, Re-exe drops by 16.93% on HumanEval, underlining the essential role of control-flow information for decompilation correctness. Removing DM leads to smaller but consistent degradations of 1.20% on Re-com, 4.67% on Re-exe and 0.36% on ES, indicating that data-level correspondences still enhance variable alignment and execution fidelity. (2) Models that integrate both CFG and DM demonstrate more robust performance

across different compiler optimization levels (O0–O3). In contrast, models with either component removed show greater performance variance and occasional regressions. This indicates that combining structural (CFG) and symbolic (DM) inductive biases enables better generalization to real-world, compiler-transformed binaries.

4.4 The Necessity of Data Flow (RQ4)

Data flow has been widely utilized in various binary analysis tasks, such as type inference[36], due to its ability to capture dynamic program behaviors and dependencies. Inspired by that, we investigate the impact of data-flow features on LLM-driven decompilation.

We design a def-use graph extraction pipeline based on Ninja [26], which operates directly on disassembled code. In the def-use graph, each node represents an assembly instruction, and edges denote data dependencies: directed links run from definition sites (where registers or memory locations are written) to use sites (where they are read), yielding an explicit data-flow view of the binary. This structure enables subsequent phases to trace data flow trajectories and perform data-flow-guided reasoning tasks (e.g., constant propagation, dead-store elimination) during decompilation.

Aligning with the CFG representation in CIW, we first structure each def-use graph (DUG) as a JSON dictionary containing two fields—“nodes” (individual assembly instructions) and “edges” (a list of directed edges)—and then append this serialized JSON string to the CIW prompt.

Table 7 reports decompilation results of general-purpose LLMs on the HumanEval with the assistance of **DUG-augmented CIW**. By comparing Table 1 and Table 7, we find that incorporating DUG does not yield significant improvements in decompilation performance. On average, the three models achieve absolute gains of 2.94% on Rec-com, 0.01% on ES and absolute decreases of 0.19% on Rec-exe. Several factors may account for this phenomenon: (1) The inherently intertwined nature of control and data flows in assembly code implies that they convey overlapping program semantics. As a result, data flows contribute only a limited orthogonal information into the decompilation process. (2) The granularity and representation of data flow features may not be optimally aligned with the learning objectives of LLMs. Specifically, the DUG may fail to capture critical contextual information necessary for accurate high-level program reconstruction, or introduce excessive noise that dilutes the model’s predictive capability. (3) The inherent limitations of the models themselves may further hinder their ability to effectively utilize data-flow information. The mechanisms required for general-purpose LLMs to handle complex data dependency patterns may remain under-activated, making it difficult to learn

⁴Ablation study on LLMs are usually conducted during inference rather than through retraining, since retraining LLMs for each ablation setting is resource-intensive — a common limitation in the research community.

⁵We conduct the corresponding ablation experiments only on samples where the number of CFG blocks is greater than one or where data sections are present.

```

1 void __thiscall std::_Fac_tidy_reg_t::~_Fac_tidy_reg_t(
2   std::_Fac_tidy_reg_t *this)
3 {
4   std::_Fac_node *v1; // ecx
5   std::_Fac_node *v2; // esi
6   while ( 1 )
7   {
8     v2 = Block;
9     if ( !Block )
10      break;
11     v1 = Block;
12     Block = *(std::_Fac_node **)Block;
13     std::_Fac_node::~_Fac_node(v1);
14     sub_402A62(v2); } }
(a)

1 void free_fac_nodes(void)
2 {
3   Fac_node *p;
4
5   while (fac_nodes)
6   {
7     p = fac_nodes;
8     fac_nodes = fac_nodes->next;
9
10    p->~Fac_node();
11    free__bench(p);
12  }
13 }
(b)

1 void __fastcall sub_23C6(_QWORD *a1)
2 {
3   *a1 = 0LL;
4   a1[1] = 0LL;
5   a1[2] = 0LL;
6   a1[3] = 0LL;
7   a1[4] = 0LL;
8   a1[5] = 0LL;
9 }
(c)

1 void init_00000001(struct stack *stk)
2 {
3   stk->top = 0;
4   stk->size = 0;
5   stk->capacity = 0;
6   stk->array = NULL;
7   stk->min = 0;
8   stk->min_capacity = 0;
9 }
(d)

```

Figure 5: (a) shows the decompilation result of function 0x4029BB in *Clever Bird.exe* produced by Hex-Rays; (b) presents the corresponding decompilation result generated by CIM-6.7B; (c) shows the decompilation result of function 0x23c6 in *revvm* produced by Hex-Rays; and (d) presents the corresponding decompilation result generated by CIM-6.7B.

the nuanced relationships between data-flow features and their corresponding high-level constructs.

Beyond the main experimental evaluation, we conduct case studies demonstrations. Due to page limitations, we present these content in the Supplementary Content A.3.

4.5 Inference Efficiency Analysis (RQ5)

To assess the practical applicability of our CIM, we conduct an efficiency analysis beyond decompilation performance on HumanEval. We evaluate computational costs in terms of inference latency and GPU memory consumption, and compare our CIM with larger LLM baselines. This analysis quantifies the computational requirements and scalability of our approach.

Specifically, CIM-1.3B requires at least 3GB GPU memory and takes 369s on 2×H100, while CIM-6.7B requires at least 14GB GPU memory and takes 959s. Under our API evaluation setting with approximately 1 Gbps bandwidth, Bailian-API-based DeepSeek-V3 and Qwen-plus take 2147s and 3734s, respectively.

The performance and efficiency analyses show that our proposed models achieve competitive or superior performance with substantially lower computational costs compared with larger baseline LLMs.

4.6 Real-world Application

To validate the practical effectiveness of our CIM, we randomly select binaries such as *Clever Bird.exe* and *revvm* from reverse engineering challenges in past CTF competitions (XNUCA 2019 and hxp CTF 2021) and perform decompilation on them (as shown in Fig. 5). From the decompiled results of functions such as ‘0x4029BB’ in *Clever Bird.exe* and ‘0x23c6’ in *revvm*, it can be observed that our decompilation-specific LLM outperforms Hex-Rays in terms of variable/function renaming, structure representation, and overall code readability.

5 Discussion

Through different-bit experiments, we find that LLM-based decompilers exhibit promising scalability and generalization in terms of development and maintenance; extending them to new compilers and architectures is essentially an engineering increment driven by data diversity.

Although our exploratory study has made some progress, several important challenges remain open for future investigation:

(1) The experiments indicate a significant drop in decompilation performance on binaries compiled with higher optimization levels. Understanding the underlying reasons behind this degradation and designing models that are more robust to aggressive compiler optimizations are promising directions for future research.

(2) We currently focus on the single-function in binaries, and in future work, we will further explore the decompilation of entire binaries.

(3) Existing works perform one-time decompilation. In practice, it is more desirable to build a self-correcting decompiler, which can refine its own decompilation results through execution feedback and comparison with the assembly.

(4) Mitigating LLM hallucinations is critical, as generated code may contain fabricated or inconsistent logic that reduces the success rate of binary decompilation.

(5) Applicability to complex binaries. Current work primarily focuses on standard binaries with available program information. Extending LLM-based decompilation to stripped, obfuscated, packed, and protected binaries remains an open challenge due to the loss of program semantics and increased analysis complexity.

6 Conclusion

Despite recent advances, end-to-end decompilation with LLMs still suffers from limited control flow awareness, weak data context modeling, and high computational costs. This paper addresses these challenges by introducing a structure-aware and data-informed decompilation scheme that integrates CFGs and data mapping tables into LLM inputs. We build a dedicated dataset and propose CIM, a lightweight domain-specific LLM, and demonstrate that it achieves state-of-the-art performance on multi-metrics while significantly reducing computational costs to enable efficient and locally deployable inference.

7 Data Availability Statement

The complete training and evaluation code and dataset, along with the optional Supplementary Content of this paper, are archived on Zenodo at <https://doi.org/10.5281/zenodo.19351453>, and also on Github at <https://github.com/LiuPeiP-CS/CodeInverter>.

Acknowledgments

Supported by Zhongguancun Laboratory. We thank the anonymous reviewers for their insightful comments and constructive suggestions.

References

- [1] Jordi Armengol-Estapé, Jackson Woodruff, and et al. 2022. ExeBench: an ML-scale dataset of executable C functions. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming* (San Diego, CA, USA) (MAPS 2022). Association for Computing Machinery, New York, NY, USA, 50–59. doi:10.1145/3520312.3534867
- [2] Jordi Armengol-Estapé, Jackson Woodruff, and et al. 2024. SLaDe: A Portable Small Language Model Decompiler for Optimized Assembly. In *Proceedings of the 2024 IEEE/ACM International Symposium on Code Generation and Optimization* (Edinburgh, United Kingdom) (CGO '24). IEEE Press, 67–80. doi:10.1109/CGO57630.2024.10444788
- [3] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).
- [4] Binutils. 2025. objdump. [Online]. <https://www.gnu.org/software/binutils/>
- [5] Georges Bossert, Frédéric Guihéry, and et al. 2014. Towards automated protocol reverse engineering using semantic information. In *Proceedings of the 9th ACM Symposium on Information, Computer and Communications Security* (Kyoto, Japan) (ASIA CCS '14). New York, NY, USA, 51–62. doi:10.1145/2590296.2590346
- [6] Ying Cao, Ruigang Liang, and et al. 2022. Boosting Neural Networks to Decompile Optimized Binaries. In *Proceedings of the 38th Annual Computer Security Applications Conference* (Austin, TX, USA) (ACSAC '22). New York, NY, USA, 508–518. doi:10.1145/3564625.3567998
- [7] Yaniv David, Uri Alon, and et al. 2020. Neural reverse engineering of stripped binaries using augmented control flow graphs. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 225 (Nov. 2020), 28 pages. doi:10.1145/3428293
- [8] DeepSeek-AI, Aixin Liu, Bei Feng, and et al. 2024. DeepSeek-V3 Technical Report. *ArXiv abs/2412.19437* (2024). <https://api.semanticscholar.org/CorpusID:275118643>
- [9] Manuel Egele, Theodoor Scholte, and et al. 2008. A survey on automated dynamic malware-analysis techniques and tools. *ACM Comput. Surv.* 44, 2, Article 6 (March 2008), 42 pages. doi:10.1145/2089125.2089126
- [10] Yunlong Feng, Bohan Li, Xiaoming Shi, Qingfu Zhu, and Wanxiang Che. 2025. ReF Decompile: Relabeling and Function Call Enhanced Decompile. *arXiv preprint arXiv:2502.12221* (2025). doi:10.48550/arXiv.2502.12221
- [11] Yunlong Feng, Dechuan Teng, Yang Xu, and et al. 2024. Self-Constructed Context Decompilation with Fined-grained Alignment Enhancement. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, Miami, Florida, USA, 6603–6614. doi:10.18653/v1/2024.findings-emnlp.385
- [12] Cheng Fu, Huili Chen, and et al. 2019. Coda: an end-to-end neural program decompiler. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA, Article 333, 12 pages. https://proceedings.neurips.cc/paper_files/paper/2019/file/093b60fd0557804c8ba0cbf1453da22f-Paper.pdf
- [13] Han Gao, Shaoyin Cheng, and et al. 2021. A lightweight framework for function name reassignment based on large-scale stripped binaries. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Virtual, Denmark) (ISSTA 2021). New York, NY, USA, 607–619. doi:10.1145/3460319.3464804
- [14] Ghidra. 2025. Ghidra Software Reverse Engineering Framework. Accessed: 2025-01-14. <https://ghidra-sre.org/>
- [15] Hex-Rays. 2025. Hex-Rays Decompiler. Accessed: 2025-01-14. <https://hex-rays.com/decompiler/>
- [16] SA Hex-Rays. 2014. IDA Pro: a cross-platform multi-processor disassembler and debugger. <https://hex-rays.com/ida-pro>
- [17] Iman Hosseini and Brendan Dolan-Gavitt. 2022. Beyond the C: Retargetable Decompilation using Neural Machine Translation. In *Proceedings 2022 Workshop on Binary Analysis Research (BAR 2022)*. Internet Society. doi:10.48550/arXiv.2212.08950
- [18] Peiwei Hu, Ruigang Liang, and et al. 2024. DeGPT: Optimizing Decompiler Output with LLM. In *Proceedings 2024 Network and Distributed System Security Symposium (2024)*. <https://api.semanticscholar.org/CorpusID>, Vol. 267622140. doi:10.14722/ndss.2024.24401
- [19] Haseeb Javed, Farman Ali, Babar Shah, and Daehan Kwak. 2025. Binary Code Analysis for Cybersecurity: A Systematic Review of Forensic Techniques in Vulnerability Detection and Anti-Evasion Strategies. *IEEE Access* 13 (2025), 167139–167164. doi:10.1109/ACCESS.2025.3610616
- [20] Nan Jiang, Chengxiao Wang, Kevin Liu, and et al. 2025. Nova: Generative Language Models for Assembly Code with Hierarchical Attention and Contrastive Learning. In *International Conference on Learning Representations*. doi:10.48550/arXiv.2311.13721
- [21] Deborah S. Katz, Jason Ruchti, and et al. 2018. Using recurrent neural networks for decompilation. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 346–356. doi:10.1109/SANER.2018.8330222
- [22] Omer Katz, Yuval Olshaker, Yoav Goldberg, and Eran Yahav. 2019. Towards Neural Decompilation. *arXiv:1905.08325 [cs.LG]* doi:10.48550/arXiv.1905.08325
- [23] Christopher Kruegel, William Robertson, and et al. 2004. Detecting Kernel-Level Rootkits Through Binary Analysis. In *Proceedings of the 20th Annual Computer Security Applications Conference (ACSAC '04)*. IEEE Computer Society, USA, 91–100. doi:10.1109/CSAC.2004.19
- [24] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, and et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (SOSP '23). Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165
- [25] Ruigang Liang, Ying Cao, and et al. 2021. Neutron: an attention-based neural decompiler. *Cybersecurity* 4 (03 2021), 5. doi:10.1186/s42400-021-00070-0
- [26] Evan Martin. 2012. Ninja: a small build system with a focus on speed. <https://ninja-build.org/>. Accessed: 2025-11-12.
- [27] OpenAI, Josh Achiam, Steven Adler, and et al. 2024. GPT-4 Technical Report. *arXiv:2303.08774 [cs.CL]* doi:10.48550/arXiv.2303.08774
- [28] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018). <https://api.semanticscholar.org/CorpusID:49313245>
- [29] Xinyu She, Yanjie Zhao, and et al. 2024. WaDec: Decompiling WebAssembly Using Large Language Model. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) (ASE '24). New York, NY, USA, 481–492. doi:10.1145/3691620.3695020
- [30] László Szekeres, Mathias Payer, and et al. 2013. SoK: Eternal War in Memory. In *2013 IEEE Symposium on Security and Privacy*. 48–62. doi:10.1109/SP.2013.13
- [31] Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. 2024. LLM4Decompile: Decompiling Binary Code with Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 3473–3487. doi:10.18653/v1/2024.emnlp-main.203
- [32] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023). doi:10.48550/arXiv.2302.13971
- [33] Wai Kin Wong, Daoyuan Wu, Huaijin Wang, et al. 2025. DecLLM: LLM-Augmented Recompileable Decompilation for Enabling Programmatic Use of Decompiled Code. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA081 (June 2025), 24 pages. doi:10.1145/3728958
- [34] Zeping Yu, Rui Cao, and et al. 2020. Order Matters: Semantic-Aware Neural Networks for Binary Code Similarity Detection. *Proceedings of the AAAI Conference on Artificial Intelligence* 34, 01 (Apr. 2020), 1145–1152. doi:10.1609/aaai.v34i01.5466
- [35] Zeping Yu, Wenxin Zheng, Jiaqi Wang, et al. 2020. CodeCMR: cross-modal retrieval for function-level binary source code matching. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 326, 12 pages. https://proceedings.neurips.cc/paper_files/paper/2020/file/285f89b802bcb2651801455c86d78f2a-Paper.pdf
- [36] Chang Zhu, Ziyang Li, Anton Xue, et al. 2024. TYGR: type inference on stripped binaries using graph neural networks. In *Proceedings of the 33rd USENIX Conference on Security Symposium* (Philadelphia, PA, USA) (SEC '24). USENIX Association, USA, Article 240, 18 pages. <https://par.nsf.gov/biblio/10585728>